420-511-VA Web Services

Week 3
Introduction to RESTful Web Services

Pravish Sainath

Course Map

Week 1

Processes communicate over sockets. We learned endpoints, TCP streams, framing, timeouts, and failures.

Week 2

HTTP standardizes messages: URIs, methods, fields, status codes, caching, and HTTPS.

Week 3

REST asks how to design the service interface so independent clients can use it coherently.

TCP socket → **HTTP message** → **RESTful service design**

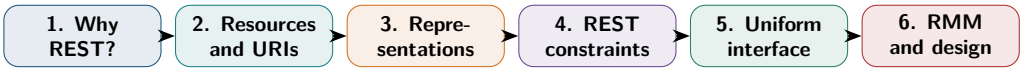
Takeaway: REST does not replace HTTP; it uses HTTP semantics to expose resources through representations.

Week 3 Learning Outcomes

By the end of this week, students should be able to:

1. explain REST as an architectural style, not a protocol or framework;
2. distinguish resources, URIs, representations, and database records;
3. model collections, members, relationships, and subresources;
4. explain REST constraints and the uniform interface;
5. choose HTTP methods and status codes for resource operations;
6. apply the Richardson Maturity Model critically;
7. transform requirements into a resource-oriented API design;
8. specify validation, errors, caching, security, consistency, and evolution concerns.

Roadmap for the Class



Examples used throughout

School portal, course registration, Instagram-like feed, ticketing/reservation, food delivery, export jobs, GitHub-style repositories, and public industry APIs.

1. Why REST?

HTTP gives messages; REST gives
architectural constraints for coherent Web APIs

The Problem REST Helps Solve

Without design discipline

- every endpoint invents its own verbs;
- errors are inconsistent;
- caches and proxies cannot help;
- clients depend on internal server structure;
- changes break clients unexpectedly.

RESTful design goal

- expose named resources;
- use uniform HTTP semantics;
- transfer representations;
- keep requests self-contained;
- allow independent evolution.

Takeaway: A good REST API feels predictable because different resources use the same interaction vocabulary.

REST Is an Architectural Style

Representational State Transfer

REST is an architectural style for distributed hypermedia systems. It is defined by constraints on component interaction, not by JSON, PHP, JavaScript, or any specific framework.

REST is not...

- a transport protocol;
- a payload format;
- a routing library;
- automatically any HTTP endpoint;
- merely database CRUD.

REST uses...

- resources;
- identifiers;
- representations;
- a uniform interface;
- stateless messages;
- cache and layered-system properties.

Same HTTP, Different API Styles

HTTP as tunnel

```
POST /api HTTP/1.1
Content-Type: application/json

{"method":"deleteStudent",
 "studentId":42}
```

One endpoint; method meaning hidden in the body.

Resource-oriented HTTP

```
DELETE /students/42 HTTP/1.1
Authorization: Bearer ...

HTTP/1.1 204 No Content
```

The target and method express intent visibly.

Takeaway: RESTful design lets clients, gateways, caches, logs, tests, and documentation understand more from standard HTTP semantics.

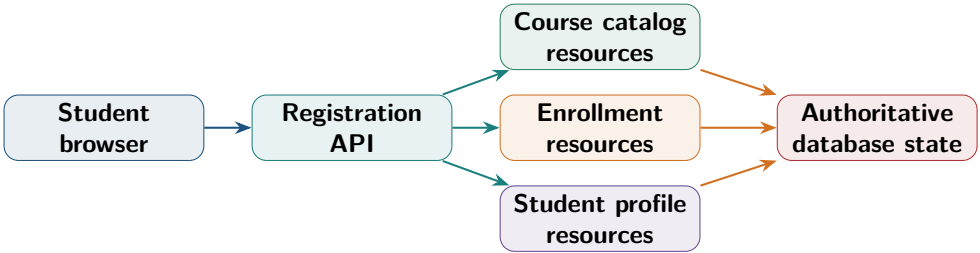
A Resource-Oriented Mental Model



Example: student, section, enrollment, seat, post, comment, order, payment

Takeaway: We first model what the service exposes, then choose names, representations, methods, and outcomes.

Everyday Example: School Portal



browser sends HTTP requests; API checks conflict, capacity, and profile rules

Resource view

`/courses/420-511`, `/sections/420-511-F26-A`, `/students/42`, and `/enrollments/8841` are resources exposed through the API contract, not necessarily database table names.

2. Resources and URIs

REST begins by naming meaningful things, not by inventing remote functions

What Is a Resource?

Resource

A resource is any concept that the service makes identifiable and interactable through its interface. It can be a document, object, collection, relationship, process, status, or view.

Objects

- student
- course
- ticket
- post

Collections

- students
- course sections
- comments
- orders

Relationships/processes

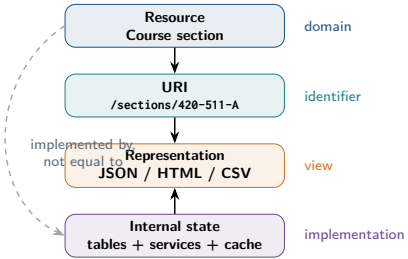
- enrollment
- follow
- reservation
- export job

Takeaway: A good resource is something users can talk about clearly, not necessarily one database row.

Resource, URI, Representation, Record

Four different concepts

- **Resource:** the meaningful thing in the API domain.
- **URI:** the identifier clients use to address it.
- **Representation:** the transferable view returned or sent.
- **Internal state:** tables, services, caches, files.
- The API contract should not expose internal structure accidentally.

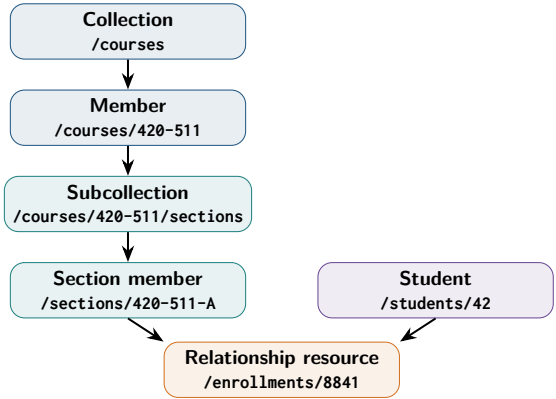


⚠ API resources should not simply expose database tables.

How to read the map

- **Collection:** a resource that lists or creates members.
- **Member:** one identified resource in a collection.
- **Subcollection:** related members scoped by a parent.
- **Relationship resource:** a link with its own ID, state, and rules.

Relationship resources carry identity, lifecycle, status, permissions, and rules.

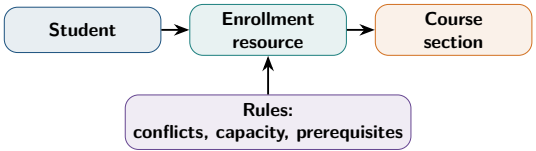


enrollment links both resources

First-Class Relationship Resources

Enrollment as resource

- has an ID;
- has status: requested, confirmed, waitlisted, dropped;
- has timestamps and audit rules;
- connects one student to one section;
- can be cancelled, queried, or waitlisted.



Better than a hidden join

Expose `/enrollments/8841` when the relationship has meaningful state and behavior.

URI Design: Nouns First

Good resource names

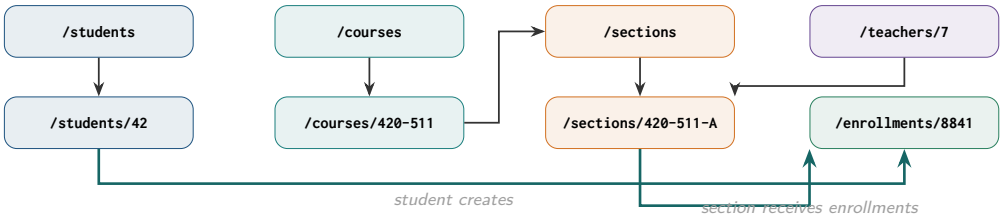
- `/students/42`
- `/sections/420-511-A`
- `/sections/420-511-A/enrollments`
- `/orders/9004`
- `/posts/771/comments`

Design smells

- `/getStudentById`
- `/deleteCourse`
- `/api/doAction`
- `/updateStudentName`
- `/runSqlQuery`

Takeaway: HTTP methods carry common operation meaning; URIs should name resources.

School Registration Resource Map



Collection resources name sets; member resources name one item; relationship resources such as enrollments connect domain objects without exposing database joins.

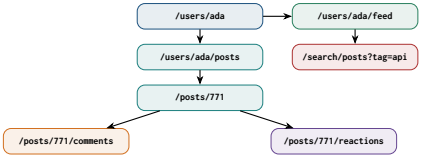
Teaching point

The API design mirrors domain concepts students understand: courses have sections, students create enrollments, teachers teach sections.

Instagram-Like Resource Map

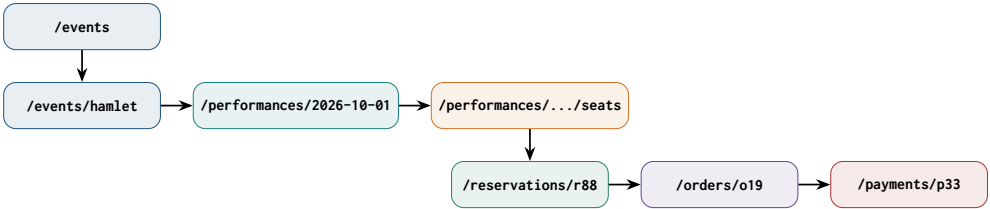
Everyday mapping

- A user has an identified profile resource.
- Posts and comments are member resources.
- A feed is a computed resource view.
- Search is also a resource view over a collection.
- Reactions/follows may be relationship resources.



Takeaway: A feed is not one table. It can depend on follows, ranking, privacy, blocks, and freshness.

Ticketing Resource Map



Scarce-resource lesson

A visible seat availability representation is informational. The authoritative reservation operation must still check and update state atomically.

3. Representations

Clients and servers exchange views of resources, not server objects themselves

Representation

A representation is a transferable view of a resource's current or intended state: JSON, HTML, XML, CSV, an image, or another media type plus metadata.



GET returns a representation; implementation state remains hidden.

Takeaway: The client never receives the server's actual object or database row; it receives a representation selected by the service.

One Resource, Many Representations

Student-facing HTML

```
GET /sections/420-511-A
Accept: text/html

HTTP/1.1 200 OK
Content-Type: text/html
```

Rendered page for humans.

API JSON

```
GET /sections/420-511-A
Accept: application/json

HTTP/1.1 200 OK
Content-Type: application/json
```

Structured data for programs.

Same resource

The course section remains the same resource, even when the server returns a human-friendly Web page, a JSON object, or a calendar feed.

Read and Write Representations

Read representation

What the server exposes when a client retrieves a resource.

- may include server-generated ID;
- may include links;
- may omit internal fields;
- may include computed status.

Write representation

What the client sends to create or change state.

- should include allowed input only;
- must be validated;
- may be partial for PATCH;
- must not control protected fields.

Takeaway: An API does not need to accept exactly the same fields it returns.

Representation Example: Enrollment

Client creates enrollment

```
POST /enrollments
Content-Type: application/json

{
  "studentId": "42",
  "sectionId": "420-511-A"
}
```

Server returns resource view

```
201 Created
Location: /enrollments/8841
Content-Type: application/json

{
  "id": "8841",
  "status": "confirmed",
  "createdAt": "2026-09-10T20:00Z"
}
```

Why not include every field?

The client should not set audit timestamps, authoritative status, or seat-count metadata. The server decides those according to business rules.

Representation Metadata

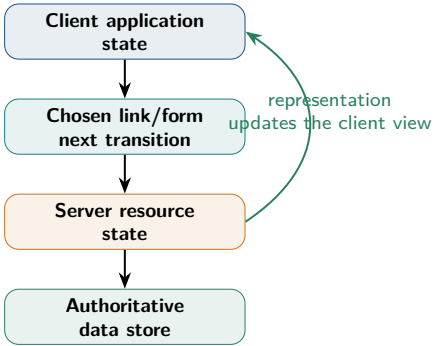
Field	Meaning	REST design use
`Content-Type`	Format returned or sent.	Tells client how to parse representation.
`Accept`	Client-preferred response format.	Supports JSON/HTML/CSV views.
`ETag`	Validator for a representation.	Enables cache validation and safe updates.
`Location`	URI of created/related resource.	Important after `201 Created`.
`Cache-Control`	Freshness and reuse policy.	Allows caching where safe.

Takeaway: A self-descriptive message carries data plus enough metadata for the receiver to interpret it correctly.

Resource State vs. Application State

Do not confuse two kinds of state

- **Client application state:** page, filters, selected tab, next link.
- **Server resource state:** enrollment status, seat count, order status.
- Representations can update what the client knows and what choices it offers.
- The authoritative resource state remains on the server side.



School portal: selected term/page = client state; actual enrollment and seat count = server state.

4. REST Architectural Constraints

The constraints are the design rules that
make the Web style scalable and evolvable

The Six REST Constraints

- **Client-server separation**
- **Stateless requests**
- **Cacheable responses**
- **Layered system**
- **Uniform interface**
- **Code-on-demand optional**

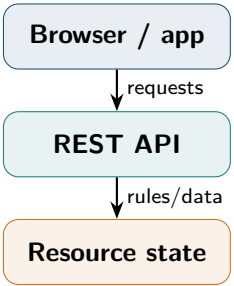
Pedagogical simplification

Each constraint limits design freedom in one way, but earns a system property such as simpler clients, scalability, evolvability, caching, or intermediary support.

Constraint 1: Client-Server

Separation of concerns

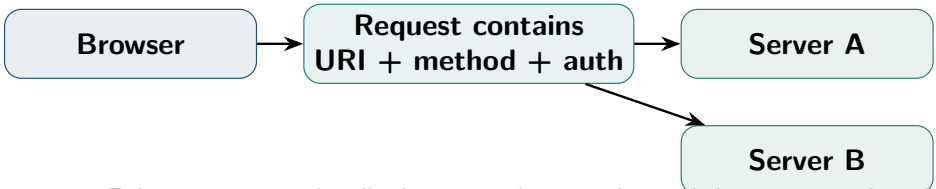
- client handles user interaction;
- server manages resources and rules;
- interface separates the two;
- each side can evolve independently.



Takeaway: A mobile app, Web page, and CLI can use the same service contract without sharing UI code.

Statelessness

Each request contains all information the server needs to understand and authorize it. The server should not depend on hidden conversational context from earlier requests.



Either instance can handle the request because the needed context travels with it.

Takeaway: Statelessness improves scalability and reliability because requests can be handled by different server instances.

Stateless Does Not Mean No State

Misconception

"A REST API cannot have users, carts, sessions, or database state."

Correct interpretation

- resource state exists on the server;
- client application state exists on the client;
- requests carry identity/context explicitly;
- server should not rely on hidden transport conversation state.

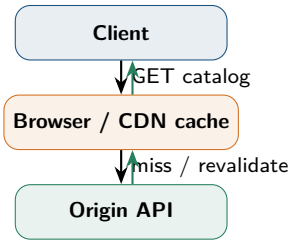
Example: A school portal may remember the logged-in user with a token/cookie, but each request must still carry enough identity and permission context.

Constraint 3: Cacheability

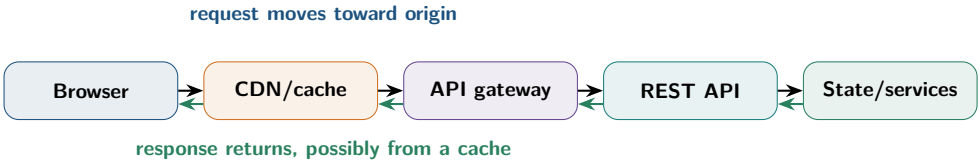
Cacheable response

A response declares whether it can be reused and under what conditions. This can reduce latency, bandwidth, and origin-server work.

- public course catalog: often cacheable;
- personal schedule: maybe private cache only;
- enrollment mutation: not cacheable as a state-changing response.



Constraint 4: Layered System



Takeaway: A client usually does not know every intermediary. The uniform interface allows intermediaries to operate without understanding private server code.

Constraint 5: Code-on-Demand

Optional REST constraint

A server may temporarily extend client functionality by sending executable code, such as JavaScript running in a browser. This is optional; an API can be RESTful without it.

Web page

A course portal sends HTML plus JavaScript. The browser executes code that calls APIs and updates the interface.

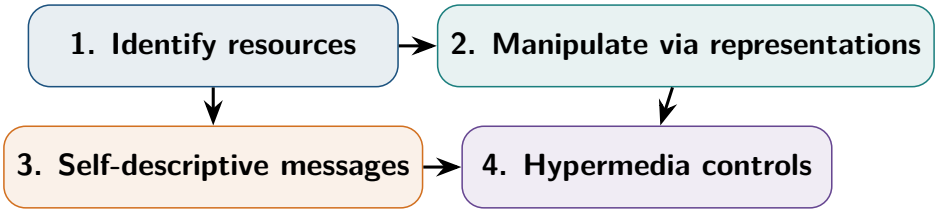
API course boundary

We mention this to complete the REST constraint list, but Week 3 focuses on Web-service interface design, not front-end JavaScript architecture.

Constraint 6: Uniform Interface

The central REST constraint

The uniform interface means clients and servers interact through a small, general set of rules rather than a new custom operation vocabulary for every service.



Takeaway: Uniformity is why HTTP methods and standard fields matter so much to REST API design.

5. The Uniform Interface

The four subconstraints that turn HTTP endpoints into a coherent REST interface

Subconstraint 1: Resource Identification

Stable identifiers

Each resource is identified by a URI. Clients should not need internal database knowledge to construct or understand core resource identifiers.

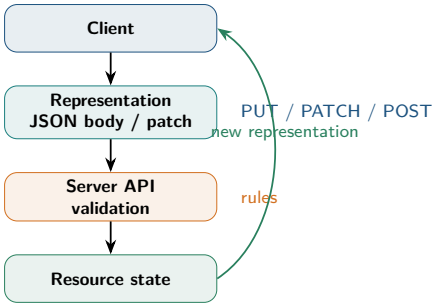
- `/students/42`
- `/courses/420-511`
- `/sections/420-511-F26-A`
- `/enrollments/8841`
- `/export-jobs/j77`

Avoid leaking internals

- `/tbl_student?id=42`
- `/join_enrollments?sid=42`
- `/storedProc/register`

The client does not run server methods

- It sends a representation describing the intended change.
- The server validates syntax, permissions, and business rules.
- If accepted, the server updates resource state.
- The response returns the new representation or a precise error.



Food delivery: send a transition representation; the server enforces allowed status changes.

Message tells its own story

- method expresses intent;
- URI identifies target;
- fields give metadata;
- media type tells parser;
- status reports outcome.

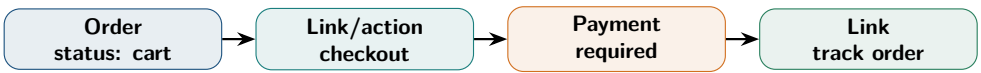
```
GET /sections/420-511-A HTTP/1.1
Accept: application/json
Authorization: Bearer ...

HTTP/1.1 200 OK
Content-Type: application/json
ETag: "s420511a-v3"
Cache-Control: private, max-age=60
```

Takeaway: Logs, proxies, tests, and client libraries benefit when meaning is visible in standard message parts.

HATEOAS in plain language

A representation can include links or actions that tell the client what transitions are currently available. The client follows affordances rather than hard-coding every next step.



Available links depend on current resource state and permissions.

Hypermedia Example


```
GET /enrollments/8841

HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": "8841",
  "status": "waitlisted",
  "section": "/sections/420-511-A",
  "links": [
    {"rel":"self", "href":"/enrollments/8841"},
    {"rel":"cancel", "href":"/enrollments/8841", "method":"DELETE"},
    {"rel":"section", "href":"/sections/420-511-A"}
  ]
}
```

Takeaway: The response communicates not only current state, but also possible next moves.

Uniform Interface: Strength and Cost

Strengths

- predictable client model;
- reusable tooling;
- compatible with intermediaries;
- easier documentation;
- better evolvability.

Costs

- less optimized than a custom protocol;
- resource modeling takes thought;
- HATEOAS can be complex;
- not every operation maps cleanly.

Takeaway: REST trades some efficiency and freedom for simplicity, visibility, and loose coupling.

6. HTTP Semantics in REST

Methods and status codes become the visible vocabulary of the API contract

Method Selection: Ask the Right Question

Method	Usual REST role	Example
`GET`	retrieve representation	`GET /sections/420-511-A`
`POST`	create subordinate resource or submit processing	`POST /enrollments`
`PUT`	replace resource at known URI	`PUT /students/42/profile`
`PATCH`	partial modification	`PATCH /orders/o19`
`DELETE`	remove resource	`DELETE /reservations/r88`
`OPTIONS`	discover permitted interaction	`OPTIONS /sections/420-511-A`

Takeaway: Do not encode every action in a URI verb when an HTTP method already expresses the operation.

Safety and Idempotence

Method	Safe?	Idempotent?	Retry intuition
`GET`	yes	yes	Retrieve again without changing requested resource.
`HEAD`	yes	yes	Same as GET metadata only.
`PUT`	no	yes	Replacing with same representation has same intended effect.
`DELETE`	no	yes	After deletion, repeating deletion keeps it deleted.
`POST`	no	no	Repeating may create another enrollment/order.
`PATCH`	no	not guaranteed	Depends on patch semantics.

⚠ Important: safe/idempotent describe intended effect; servers may still log or audit requests.

POST, PUT, PATCH: Common Confusion

POST creates subordinate

```
POST /enrollments

{"studentId":"42",
 "sectionId":"A"}
```

Server chooses URI.

PUT replaces known target

```
PUT /profiles/42

{"displayName":"Ada",
 "timezone":"America/Montreal"}
```

Same request repeated: same intended state.

PATCH modifies part

```
PATCH /orders/o19

{"status":"cancelled"}
```

Partial change; define rules.

Essential Status Codes for REST Design

Code	Use	Example
`200 OK`	successful request with representation	GET section details
`201 Created`	new resource created, usually with `Location`	POST enrollment created
`202 Accepted`	accepted for later processing	export job started
`204 No Content`	success, no body	DELETE reservation
`400 Bad Request`	malformed request	invalid JSON syntax
`401 Unauthorized`	authentication needed/invalid	missing token
`403 Forbidden`	authenticated but not allowed	not own enrollment
`404 Not Found`	no current target resource	unknown section
`409 Conflict`	request conflicts with state	seat already reserved
`412 Precondition Failed`	failed validator precondition	stale ETag update
`415 Unsupported Media Type`	unacceptable request format	XML sent where JSON required
`422 Unprocessable Content`	syntactically valid but semantic validation failed	prerequisite missing

Standard Error Representation

Problem Details

RFC 9457 defines a standard shape for HTTP API error details using `application/problem+json`.

```
HTTP/1.1 409 Conflict
Content-Type: application/problem+json

{
  "type": "https://portal.example.edu/problems/section-full",
  "title": "Section is full",
  "status": 409,
  "detail": "No confirmed seats remain in 420-511-A.",
  "instance": "/enrollment-requests/r892"
}
```

Takeaway: Use machine-readable errors so clients can react, not merely display vague prose.

Long-Running Work: Job Resource

Start work

```
POST /grade-export-jobs

HTTP/1.1 202 Accepted
Location: /grade-export-jobs/j77
```

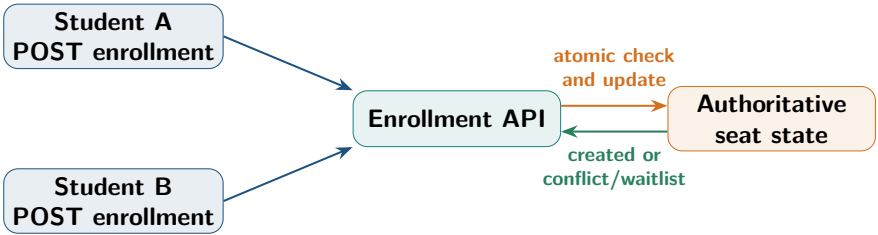
Poll job resource

```
GET /grade-export-jobs/j77

HTTP/1.1 200 OK
{
  "status": "running",
  "progress": 60
}
```

Why this is RESTful

Instead of holding one HTTP request open for minutes, the API creates a job resource with its own state and lifecycle.



A cached availability page is informational. The mutation must be decided against authoritative state.

Takeaway: RESTful APIs still need strong business rules for races, retries, duplicates, and conflicts.

Idempotency Keys for Retried POSTs

Risk

A client sends `POST /orders`, the server creates the order, but the response is lost. A blind retry can create a duplicate.

```
POST /orders
Idempotency-Key: 3e48-a91...
Content-Type: application/json

{"cartId":"c55"}
```

The server can recognize a retry of the same operation.

Takeaway: HTTP method semantics guide retry behavior, but application-level identifiers are often needed for non-idempotent operations.

7. Richardson Maturity Model

A teaching ladder for increasingly resource-oriented HTTP APIs

The Richardson Maturity Model



Not Fielding's formal REST definition. Useful as a classroom maturity ladder.

Takeaway: Many industry APIs aim for strong Level 2 with selective hypermedia.

Level 0: HTTP as a Tunnel


```
POST /api
Content-Type: application/json

{"action":"getSection",
 "id":"420-511-A"}
```

Characteristics

- one endpoint;
- one HTTP method;
- custom body verbs;
- HTTP semantics mostly ignored;
- hard for intermediaries and tools.

```
POST /sections/420-511-A
Content-Type: application/json

{"action":"get"}
```

Improvement

The API now identifies different resources, but still may tunnel all operations through POST and custom action bodies.

Takeaway: Level 1 has nouns, but not yet the full HTTP method vocabulary.

```
GET /sections/420-511-A

200 OK
Content-Type: application/json

POST /enrollments

201 Created
Location: /enrollments/8841
```

Common target level

- resources are distinct;
- HTTP methods carry intent;
- status codes carry outcomes;
- errors are standardized;
- cache and validation can work.

```
GET /orders/o19

{
  "id":"o19",
  "status":"ready_for_payment",
  "total":"48.50 CAD",
  "links":[
    {"rel":"self", "href":"/orders/o19"},
    {"rel":"pay", "href":"/orders/o19/payment", "method":"POST"},
    {"rel":"cancel", "href":"/orders/o19", "method":"DELETE"}
  ]
}
```

Takeaway: Level 3 responses teach clients the currently available transitions.

How to Use RMM Critically

Useful for students

- shows progression;
- contrasts RPC-style with resource-style;
- highlights HTTP method/status usage;
- explains hypermedia as a next step.

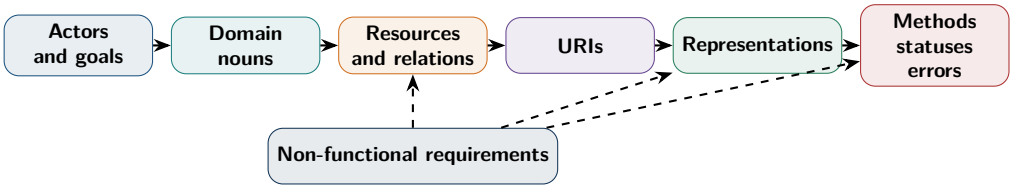
Limitations

- not the original REST definition;
- Level 2 is not automatically REST;
- Level 3 is not always worth the added complexity;
- design quality still matters.

8. REST Design Process

From requirements to resources, representations, methods, errors, and acceptance criteria

Design Pipeline



Takeaway: Do not begin with URLs. Begin with the user's goals and domain concepts.

Actor	Goal	API implication
Student	Find open sections and enroll.	searchable sections, enrollment creation, validation errors.
Teacher	View class list.	authorized roster resource, privacy rules.
Registrar	Override capacity.	privileged transition, audit logging.
System job	Export grades.	asynchronous job resource and status.

Requirement style

"A student can request enrollment in a published section if prerequisites, schedule conflicts, and seat capacity permit it."

Functional vs. Non-Functional Requirements

Functional

What the system does.

- list course sections;
- create an enrollment;
- cancel a reservation;
- export a grade report.

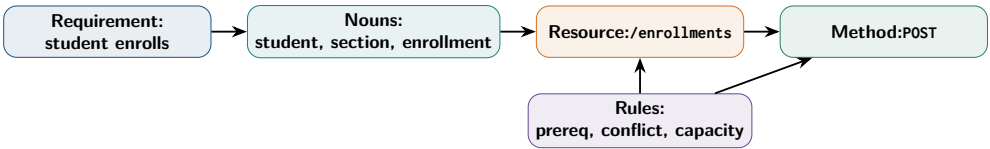
Non-functional

How well and under what constraints.

- authorization required;
- response under 500 ms;
- cache catalog for 5 minutes;
- audit every override;
- preserve compatibility.

Takeaway: A REST design spec must include both behavior and quality constraints.

From Requirements to Resources



Result

The student does not call `/runEnrollmentAlgorithm`; the student creates an enrollment request/resource that the server validates.

School Registration: Endpoint Slice

```
GET /sections?course=420-511&term=F26

200 OK
Content-Type: application/json
Cache-Control: public, max-age=300
```

```
POST /enrollments
Content-Type: application/json

{"studentId":"42", "sectionId":"420-511-A"}
```

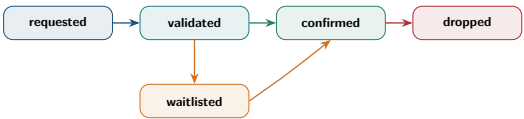
Design choices

- search sections through collection query;
- create enrollment as first-class resource;
- cache public catalog, not private decisions;
- use precise conflict and validation errors.

Resource Lifecycle

Example: enrollment request

- A resource often has allowed states, not just CRUD existence.
- Methods should trigger allowed transitions.
- Errors should explain why a transition is unavailable now.



Takeaway: Design methods and errors around allowed state transitions, not just CRUD verbs.

Validation and Business Rules

Rule	Meaning	Likely response
Malformed JSON	Body cannot be parsed.	`400 Bad Request`
Unknown section	Target/resource not found.	`404 Not Found`
Missing prerequisite	Request syntactically valid, business rule fails.	`422 Unprocessable Content`
Section full	Conflicts with current state.	`409 Conflict` or waitlist creation
Student not owner	Authenticated but lacks permission.	`403 Forbidden`
Stale edit	Client tried update with old validator.	`412 Precondition Failed`

Takeaway: Validation outcomes should be part of the API contract, not an afterthought.

Non-Functional Requirements Checklist

Security/privacy

- auth required;
- role/owner checks;
- hide sensitive fields.

Consistency

- atomic seat updates;
- conflict rules;
- idempotency policy.

Performance

- pagination;
- caching;
- response budgets.

Reliability

- timeouts;
- retries;
- async jobs.

Evolution

- versioning policy;
- additive changes;
- deprecation.

Observability

- request IDs;
- logs;
- audit trail.

Traceability Matrix

Requirement	Resource	Interaction	Acceptance evidence
Search sections	`/sections`	`GET ?course=420-511`	returns matching F26 sections
Enroll student	`/enrollments`	`POST` with student and section	creates or reports precise reason
Prevent overfill	section state	atomic check/update	two simultaneous requests do not exceed capacity
Cancel enrollment	`/enrollments/id`	`DELETE` or transition	status changed and audit recorded
View roster	`/sections/id/roster`	authorized `GET`	teacher only; student list hidden otherwise

Takeaway: A requirement should trace to a resource, interaction, and testable outcome.

REST Design Smells

Smells

- verbs in every URI;
- all operations use POST;
- database table names exposed directly;
- status code always 200;
- errors are only strings;
- no pagination on collections;
- authz not part of design.

Improvements

- model resources first;
- use methods correctly;
- choose meaningful status codes;
- use problem details;
- document validation;
- plan caching and pagination;
- specify ownership/role checks.

9. REST in Today's API Landscape

REST is important, but not the only style used by modern systems

REST vs. Other API Styles

Style	Strength	Typical fit
REST/HTTP	resource-oriented, cacheable, simple tooling, human-readable.	public Web APIs, CRUD/resource domains, broad interoperability.
RPC over HTTP	direct operation calls.	internal services with action-focused APIs.
GraphQL	client selects precise fields from a graph.	complex read views across related objects.
gRPC	efficient typed contracts over HTTP/2.	internal high-performance service-to-service calls.
Event/message APIs	asynchronous decoupling.	notifications, background processing, integration workflows.

Takeaway: Choose an API style based on constraints and goals; do not force everything into REST vocabulary.

Industry API Reality

Common practice

- strong Level 2 style;
- selected hypermedia links;
- OpenAPI descriptions;
- JSON representations;
- OAuth/bearer auth;
- idempotency keys for risky POSTs.

Explore later

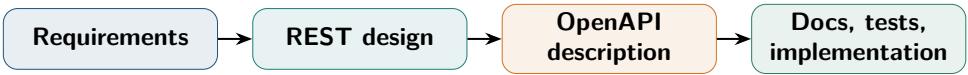
- GitHub REST API;
- MediaWiki REST API;
- Stripe API design patterns;
- Google API design guidance;
- Zalando REST guidelines.

Week 3 Design Checklist

- Have we identified domain resources?
- Are collections and members named consistently?
- Are relationships first-class where needed?
- Are representations separated from database internals?
- Are methods chosen using HTTP semantics?
- Are common status codes planned?
- Are validation and errors documented?
- Are caching rules clear?
- Are auth/authz requirements visible?
- Are conflict/idempotency rules specified?
- Are lifecycle transitions understandable?
- Can each requirement be tested?

Takeaway: A REST design is not finished until a client developer can predict how to use it and how it fails.

Transition to Week 4: OpenAPI



Next week

OpenAPI turns the REST design into a machine-readable contract: paths, operations, parameters, schemas, responses, examples, and security schemes.

10. Review and References

Use these questions and sources to consolidate the Week 3 model

Review Questions

1. Why is REST not the same thing as HTTP or JSON?
2. What is the difference between a resource and a representation?
3. When should a relationship become its own resource?
4. Why is statelessness compatible with login and sessions?
5. What does the uniform interface buy us?
6. Why is `POST /enrollments` better than `/registerStudentInCourse`?
7. What does Level 2 in the Richardson Maturity Model add over Level 1?
8. How should a design spec represent validation and conflict behavior?

Core References

- Fielding dissertation, Chapter 5: REST
- RFC 9110: HTTP Semantics
- Richardson Maturity Model, Martin Fowler
- Google AIP-121: Resource-oriented design
- Google API Design Guide
- Zalando RESTful API Guidelines
- Microsoft REST API Guidelines
- RFC 9457: Problem Details for HTTP APIs

APIs to Explore

- GitHub REST API documentation
- MediaWiki REST API
- Stripe API docs
- JSON:API specification
- OpenAPI learning site
- MDN HTTP guide

Suggested exploration task

Pick one endpoint from GitHub or MediaWiki and identify its resource, URI, method, representation, status codes, and whether it looks closer to RMM Level 2 or Level 3.